

10x20
→ 19500

46

→ 15 → 80

DAC

47

algorithm

5-10

Lec_48: Sorting Algorithms

Sorting

2nd: which sorting uses DAC
→ merge, quick

- ✓ Merge Sort
- ✓ Quick Sort
- ✓ Bubble sort
- ✓ Selection Sort
- ✓ Insertion Sort
- ✓ Heap Sort

→ $n \log n$

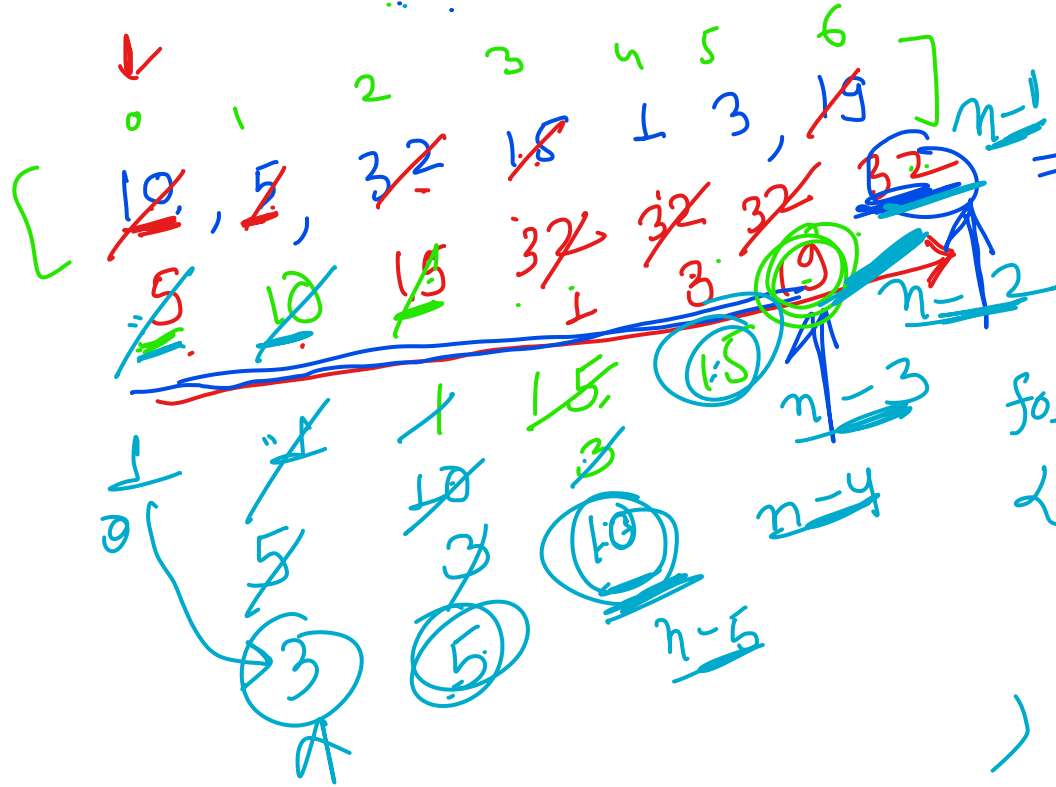
→ In DAC, already done
→ $n \log n$, n^2

→ $O(n)$
→ master

$O(n^2)$
→ each case

§ 3.1.1

array = [10, 5, 32, 15, 7, 3, 19] $\Rightarrow$ [10, 5, 10, 15, 19, 32]



```

n-1
=> maxm element at the
n-1th Index in
first pass
for(i=0; i<=n-1; i++)
{
    for(j=0; j<=n-i; j++)
    {
        if (a[j] > a[j+1])
        {
            swap(a[j], a[j+1])
        }
    }
}

```

```

Worst
for (i=0; i < n-1; i++)
{
    for (j=0; j < n-i; j++)
    {
        if (a[j] > a[j+1])
        {
            swap(a[j], a[j+1])
        }
    }
}

```

$$\frac{n + n-1 + n-2 + n-3 + \dots + 1}{2} = O(n^2)$$

Worst case Time complexity = $O(n^2)$

$\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 10 & 5 & 6 & 9 & 3 & \end{matrix}$
 $n=5$

```

for (i=0; i < 4; i++)

```

for $i=0$ $j=0$ to $j < n-1 = 4$
 $j=0$ to $j < n-1 = 4$
 $j=0$ to $j < n-2 = 3$
 $j=0$ to $j < n-3 = 2$
 $j=0$ to $j < 1 = 1$
 $j=0$ to $j < 0 = \text{X}$

→ First pass of Bubble Sort can give you first $n-1$ largest elements

#

Sort the array using Bubble Sort

main channel

1st pass → 10, 1, 9, 19, 3, 32, 6, 61, 100, 13, 100
1, 10, 9, 3, 19, 6, 32, 1, 13, 100

Result after 1st pass of Bubble Sort

2nd pass { 1, 9, 10, 3, 19, 6, 32, 61, 13, 100 }
3, 10, 9, 13, 61, 100

3rd pass 1, 9, 10, 6, 19, 32, 13, 61, 100
3, 9, 6, 10, 13, 32, 61, 100

4th pass 1, 3, 9, 6, 10, 19, 13, 32, 61, 100
6, 9, 13, 19, 32, 61, 100

5th pass 1, 3, 6, 9, 10, 13, 19, 32, 61, 100
1, 3, 6, 9, 10, 13, 19, 32, 61, 100

$x = 0$
 $y = 1$

$T.C = O(n^2)$

Worst case = $O(n^2)$

Count = no. of swaps

Count = 0

1, 3, 6, 9, 10, 13

```
for (i = 0; i < n - 1; i++)  
{  
    for (j = 0; j < n - i - 1; j++)
```

```
        if (a[j] > a[j + 1])  
        {  
            swap(a[j], a[j + 1])  
            count++  
        }  
    }
```

modified Bubble sort:

If given array and we have to sort it with bubble sort then maintain a variable count that can count no. of swaps in each iteration. If no. of swaps are 0 then that is already sorted or almost sorted.

if (count == 0)
break

[2, 4, 6, 9, 10, 13]

Swaps = 0

1st pass $\Rightarrow$ 2, 4, 6, 9, 10, 13
2nd pass $\Rightarrow$ 2, 4, 6, 9, 10, 13

that is already sorted or almost sorted

```

for(i=0; i<n-1; i++)
{
    for(j=0; j<n-i; j++)
    {
        if(a[j] > a[j+1])
        {
            swap(a[j], a[j+1])
        }
    }
    if(i==0)
    {
        return;
    }
}

```

Best case = $O(n)$
 worst case = $O(n^2)$

Best case: only one
 time loop is entered
 $T_c = O(n)$

- $T_c = O(n^2)$ always
 as it is modified at
 all steps
- Stable algorithm
- Inplace algorithm